Experiment No : 3

Title: Perform Bubble sort, Selection sort, Insertion, Quick and Merge sort.

Name of Student: Shamiksha Chaudhari
Class: SY **Div:** SY1 **PRN No:** B25ET1005 **Batch:** A1

Program:

```
#include <stdio.h>
#include <stdlib.h>

/* Function Prototypes */
int enterarray(int input[]);
void bubblesort(int input[] , int output[] , int n);
void selectionsort(int input[] , int output[] , int n);
void swap(int *a, int *b);
int partition(int input[] , int l , int h);
void quicksort(int input[] , int output[] , int l , int h);
void insertionsort(int input[] , int output[] , int n);
void mergesort(int input[] , int output[] , int l , int h);
void merge(int output[] , int l , int mid , int h );


int main()
{
   int choice , i =0 , n = 0 ;
   int arr[100] , bs[100] , ss[100] , qs[100] , is[100] , ms[100];
   //Initialize();
  do
   {
      printf("\n\t\t\t# SORTING OPERATIONS #             ");
      printf("\n----------------------------------------");
      printf("\n1. Insert an array");
      printf("\n2. Bubble sort the array");
      printf("\n3. Selection sort the array");
      printf("\n4. Partition sort the array");
      printf("\n5. Insertion sort the array");
      printf("\n6. Merge sort the array");
      printf("\n7. Exit");
```

```
printf("\n\nEnter your choice : ");
scanf("%d", &choice);

switch(choice)
{
    case 1:
        n = enterarray(arr);
        break;

    case 2:
        bubblesort(arr, bs, n);
        printf("The bubble sorted array is: ");
        for (i = 0; i < n; i++)
        {
            printf("%d ", bs[i]);
        }
        printf("\n");
        break;

    case 3:
        selectionsort(arr, ss , n);
        printf("The selection sorted array is: ");
        for (i = 0; i < n; i++)
        {
            printf("%d ", ss[i]);
        }
        printf("\n");
        break;

    case 4:
      if (n > 0)
      {
      quicksort(arr, qs, 0, n - 1);
      }
      printf("The quick sorted array is: ");
      for (i = 0; i < n; i++)
      {
          printf("%d ", qs[i]);
      }
      printf("\n");
```

```
            break;

        case 5:
            insertionsort(arr, is , n);
            printf("The insertion sorted array is: ");
            for (i = 0; i < n; i++)
            {
                printf("%d ", is[i]);
            }
            printf("\n");
            break;

        case 6:
            for (i = 0; i < n; i++)
            {
              ms[i] = arr[i];
            }
            mergesort(arr, ms , 0 , n-1);
            printf("The merge sorted array is: ");
            for (i = 0; i < n; i++) {
                printf("%d ", ms[i]);
            }
            printf("\n");
            break;


        case 7:
            return 0;

        default:
            printf("\nInvalid Choice!");
      }
   }while(1);
return 0;

}

int enterarray(int input[])
{
    int i , n;
```

```
    printf("enter length of the array : ");
    scanf("%d" ,&n);
    printf("enter elements of the array : ");
    for (i=0;i<n;i++)
    {
        scanf("%d" , &input[i]);
    }
 return n;
}

void bubblesort(int input[] , int output[] , int n)
{
    int i , j , temp ;
    for (i = 0; i < n; i++)
    {
        output[i] = input[i];
    }
    for(i = 0 ; i < n-1 ; i++)
    {
        for(j = 0 ; j < n-1-i ; j++)
        {
            if (output[j] > output[j+1])
            {
               temp = output[j];
               output[j] = output[j+1];
               output[j+1] =  temp ;
            }
        }
    }
}

void selectionsort(int input[] , int output[] , int n)
{
    int i , j , minindex , temp ;
    for (i = 0; i < n; i++)
    {
        output[i] = input[i];
    }
    for (i = 0 ; i < n - 1 ; i++)
    {
```

```
        minindex = i;
        for (j = i + 1 ; j < n ; j++)
        {
            if (output[j] < output[minindex])
            {
                minindex = j;
            }
        }
        temp = output[i];
        output[i] = output[minindex];
        output[minindex] = temp;
    }
}

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int input[] , int l , int h)
{
    int pivot = input[h];
    int i = l - 1 , j;
    for (j = l ; j < h ; j++)
    {
        if ( input[j] < pivot )
        {
            i++;
            swap( &input[i] , &input[j]);
        }
    }
    swap( &input[i+1] , &input[h]);
    return (i+1) ;
}

void quicksort(int input[] , int output[] , int l , int h )
{
    int i ;
```

```
 if(l == 0 && h >= 0 && input != output )
   {
      for (i = 0; i <= h; i++)
      {
         output[i] = input[i];
      }
   }
   if ( l < h )
   {
      int pi = partition(output , l , h);
      quicksort(output , output, l , pi - 1);
      quicksort(output , output , pi + 1 , h);
   }
}

void insertionsort(int input[] , int output[] , int n)
{
  int i , j , key ;
  for ( i = 0; i < n; i++)
  {
    output[i] = input[i];
  }
  for ( i = 1 ; i < n ; i++ )
  {
     key = output[i];
     j = i - 1 ;
     while ( j >= 0 && output[j] > key )
     {
        output[j+1] = output[j];
        j--;
     }
     output[j+1] = key ;
  }
}

void mergesort(int input[] , int output[] , int l , int h )
{
  if (l < h)
  {
     int mid = (l + h )/ 2;
     mergesort(output , output , l , mid );
     mergesort(output , output , mid + 1 , h  );
     merge(output , l , mid , h );
```

```
  }
}

void merge(int output[] , int l , int mid , int h )
{
    int i = l;
    int j = mid +1 ;
    int k = l;
    int b[100];
    while( i <= mid && j <= h)
    {
       if (output[i] <= output[j])
       {
        b[k] = output[i];
        i++;
       }
       else
       {
        b[k] = output[j];
        j++;
       }
       k++;
    }
    while( i <= mid)
    {
        b[k] = output[i];
        i++;
        k++;
    }
    while( j <= h)
   {
       b[k] = output[j];
       j++;
       k++;
   }
    for (i = l; i <= h; i++)
    {
        output[i] = b[i];
    }
}
```

Output :

```
            # SORTING OPERATIONS #
----------------------------------------
1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Partition sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit

Enter your choice : 1
enter length of the array : 5
enter elements of the array : 30 50 10 40 20
```

```
            # SORTING OPERATIONS #
----------------------------------------
1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Partition sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit

Enter your choice : 2
The bubble sorted array is: 10 20 30 40 50
```

```
            # SORTING OPERATIONS #
----------------------------------------
1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Partition sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit

Enter your choice : 3
The selection sorted array is: 10 20 30 40 50
```

```
            # SORTING OPERATIONS #
----------------------------------------
1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Partition sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit

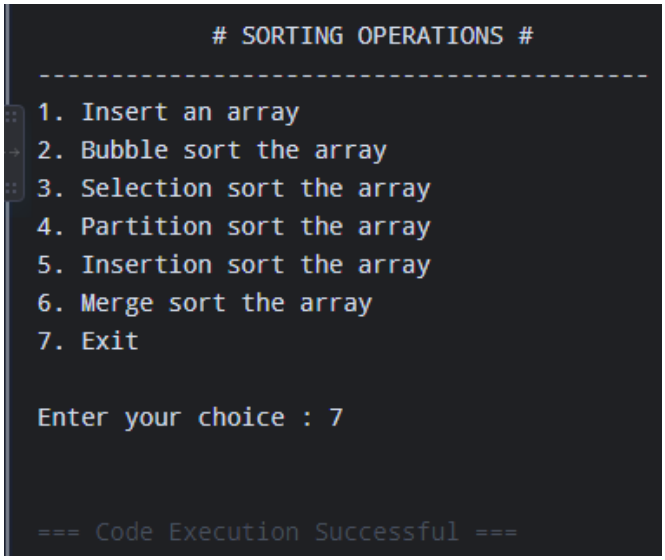
Enter your choice : 4
The quick sorted array is: 10 20 30 40 50
```

```
            # SORTING OPERATIONS #
----------------------------------------
1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Partition sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit

Enter your choice : 5
The insertion sorted array is: 10 20 30 40 50
```

```
            # SORTING OPERATIONS #
----------------------------------------
1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Partition sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit

Enter your choice : 6
The merge sorted array is: 10 20 30 40 50
```


SORTING OPERATIONS
1. Insert an array
2. Bubble sort the array
3. Selection sort the array
4. Partition sort the array
5. Insertion sort the array
6. Merge sort the array
7. Exit
Enter your choice : 7
=== Code Execution Successful ===